A Reproducible Firmware Security Analysis of OpenWRT 22.03.7 on the ath79/MIPS Platform

Ali Murtaza Bhutto
Independent Researcher
ORCID: 0009-0007-2787-943X
alibhutto101112@gmail.com

***Abstract*—Embedded network appliances are routinely shipped with months- or years-old upstream dependencies, yet the chain of evidence from a firmware image to a concrete vulnerable package is rarely reproducible. We describe a fully scripted pipeline that, given a single firmware binary, locates and extracts its SquashFS root filesystem, enumerates filesystem-level credential and configuration exposure, parses the package manager's status manifest to extract exact upstream versions, correlates those versions against the National Vulnerability Database, and runs a headless binary analysis against selected executables to surface high-risk libc call sites. We apply the pipeline to the OpenWRT 22.03.7 release image for the TP-Link Archer C7 v2 (ath79, mips_24kc). On this image the pipeline parses 144 opkg packages and reports 16 NVD-confirmed CVEs across 9 mapped components (8 critical, 5 high, 2 medium, 1 low), surfaces 23 filesystem-level pattern matches across four categories, and recovers 256 suspicious in-binary call sites across 13 libc families in the 323 KiB MIPS busybox binary. The pipeline is open, input-pinned by SHA-256, and gated by bandit, pip-audit, semgrep, hadolint, and shellcheck.**

***Index Terms*—firmware analysis; reproducible research; IoT security; CVE correlation; Ghidra; OpenWRT; SquashFS; MIPS.**

I. Introduction

The IoT security community has converged on a small handful of canonical risks: weak authentication, exposed services, hard-coded secrets, and outdated software components [1], [2]. ENISA's good-practices baseline emphasises the same recurring defects [3]. Despite this convergence, audits of consumer-grade network equipment still routinely rely on bespoke glue between extraction tools, manual enumeration of dependency versions, and ad-hoc CVE lookups. The chain of custody from a firmware blob to a concrete vulnerability claim is therefore opaque and hard to reproduce.

This paper presents a pipeline that closes that gap on a single representative target. The pipeline is scripted end-to-end; every input is content-addressed by SHA-256; every intermediate and final artefact is committed under `results/`; and every script is covered either by tests or by static-analysis gates. Re-running the pipeline on a fresh host reproduces the numbers reported below.

We deliberately scope the work to a single firmware image — the OpenWRT 22.03.7 release for the TP-Link Archer C7 v2 — because the goal is depth and reproducibility of methodology rather than breadth of coverage. OpenWRT is an appropriate target: it ships a complete upstream-package manifest [4], which makes dependency-version extraction tractable, and is widely deployed on consumer hardware.

II. Pipeline

The pipeline has four stages, each driven by a single script. Figure-free, but the data flow is linear: *firmware.bin* → *squashfs-root* → (*pattern_hits.json*, *firmwalker.txt*, *cve_correlate.json*, *ghidra_report.json*).

A. Extraction

The release image is a 6,488,389-byte concatenation of a u-boot loader, a Linux kernel blob, and a SquashFS payload. The extraction script invokes binwalk to identify the SquashFS signature offset, carves the corresponding byte range with `dd`, and unpacks it with `unsquashfs`. The result is a 1,371-inode root filesystem (1,046 files, 122 directories, 203 symlinks). The extracted filesystem is then exposed to the downstream steps via `work/squashfs-root/`.

We use the Rust rewrite of binwalk (3.1.0) rather than the legacy Python implementation; the PyPI `binwalk` package name is unrelated and unmaintained at the time of writing.

B. Filesystem enumeration

Enumeration combines two passes. The first invokes upstream *firmwalker* from its own install directory so its relative `data/` references resolve. The second is a curated regex scanner that searches text-shaped files (configs, scripts, keys, PEM material) for eight patterns: private-key headers, password assignments, telnet references, OpenSSL invocations, hard-coded HTTP URLs, default admin login patterns, blank root password entries, and 16+-character API-token-shaped assignments. Pattern hits are emitted as line-anchored JSON records.

On the target image the curated scanner records 23 hits across four categories: one `password_assignment`, one `openssl_invocation`, twenty `hardcoded_url_http`, and one `root_passwd_blank`. The HTTP-URL hits are dominated by upstream package references in scripts and OPKG metadata; the root-password-blank hit is the expected stock OpenWRT `root::` entry that the device prompts the operator to change on first boot.

TABLE I
NVD FINDINGS FOR OPENWRT 22.03.7 (ATH79).

Severity	Count
Critical	8
High	5
Medium	2
Low	1
Total	16

C. Dependency / CVE correlation

The OpenWRT image carries a full opkg manifest at `/usr/lib/opkg/status` [4]. We parse the manifest into `(Package, Version, Section)` triples and normalise the version by stripping packaging revisions (`-N`, `+TAG-N`). We then consult a conservative mapping table from opkg package name to NVD vendor/product pair (busybox, dropbear, OpenSSL variants, uhttpd, dnsmasq, the wpad/hostapd family, curl/libcurl, the Linux kernel, ppp, and samba), and query the NVD 2.0 REST API [5] with the constructed CPE 2.3 string.

The script respects the unauthenticated NVD rate limit (one query per six seconds) and caches each response keyed by a SHA-256 of the CPE so repeat runs are network-free. Output comprises a machine-readable `cve_correlate.json` and a severity-grouped `findings.md`; both are committed.

On the OpenWRT 22.03.7 image the parser recovers 144 packages, of which 9 have an entry in the NVD mapping table. The query returns 16 CVEs in total: 8 Critical, 5 High, 2 Medium, and 1 Low. The critical bucket is dominated by the dnsmasq heap-overflow cluster (CVE-2021-45951 through CVE-2021-45957) and the busybox awk out-of-bounds writes published as CVE-2022-48174.

D. Headless binary analysis

The fourth stage drives Ghidra 12.0.4 headlessly via PyGhidra, which replaced the bundled Jython runtime in the 12 release. The analysis script collects four payloads from the active program: external imports, exported entry points, suspicious libc-family call sites, and printable strings of length ≥ 12. For the suspicious-call walk, the script indexes every program function that is a thunk, identifies thunks whose ultimate target name matches a curated list (`system`, `popen`, `execve`, `execvp`, `execl`, `execlp`, `strcpy`, `strcat`, `sprintf`, `memcpy`, `setuid`, `setgid`, `seteuid`), then collects all unique in-binary callers of each thunk via the reference manager. This indirection is necessary because external symbols in Ghidra resolve to a synthetic `EXTERNAL:` address with no direct in-binary references; the real call sites reach them through PLT/GOT-style thunks at concrete addresses.

Applied to the 323 KiB MIPS busybox binary extracted from `/bin/busybox`, the analysis recovers 328 imports, 20 exports, and 256 unique suspicious call sites across all 13 categories. The heaviest contributors are `memcpy` (112 sites) and `strcpy` (79 sites), consistent with busybox's all-applets-in-one C codebase.

III. REPRODUCIBILITY

We treat reproducibility as a first-class property of the pipeline, not an afterthought.

Input pinning. The firmware image is committed with a `sha256sum` of `0ca4fe70e...99b677713`; the README and the replication guide both record the exact byte length (6,488,389) and the expected SquashFS offset.

Tool pinning. `requirements.txt` pins every Python dependency to a minor version range. binwalk is pinned at 3.1.0, Ghidra at 12.0.4, and Temurin JDK at 21.0.5. The Docker image captures the lighter-weight legs hermetically; an `install_ghidra.sh` helper handles the heavy binary analysis leg.

Output committal. Every artefact under `results/` in the repository is the verbatim output of a real pipeline run on the committed firmware sample; nothing is hand-edited.

Gates. The repository is gated by bandit on the Python sources, pip-audit on the dependency tree, semgrep with the `p/python` and `p/security-audit` rule packs, hadolint on the Dockerfile, and shellcheck on every shell script. All gates pass with zero findings; two `bandit nosec` annotations document the intentional subprocess use that drives firmwalker.

Tests. The `cve_correlate` and `enumerate` modules carry 22 pytest cases covering the opkg parser, version normaliser, CPE constructor, NVD client (mocked, including 404 and API-key paths), severity extractor, markdown renderer, pattern scanner, text-file walker, and tree renderer. Two of the tests caught real regex bugs during authoring — a PEM-header pattern that omitted the literal word *PRIVATE* in front of *KEY*, and an API-token pattern that did not tolerate a closing quote between the key name and the assignment separator. Both were fixed before the live re-run, and the live counts on the target image were unaffected.

IV. LIMITATIONS

The CVE correlator queries NVD by exact-version CPE; it will miss findings whose CPE entries omit the patch component or that index the bug under a different vendor name. Our mapping table covers the OpenWRT subset of upstream components we expect on a typical 22.03 image; broader coverage would need either an automated CPE search or a richer mapping resource. The binary analysis stage runs on a single executable per invocation; running it across an entire `/bin`/`/usr/bin`/`/usr/sbin` tree is practical but not yet scripted.

V. CONCLUSION

A scripted, content-addressed pipeline turns firmware audit from a sequence of artisanal steps into a reproducible artefact. On the OpenWRT 22.03.7 release image for the Archer C7 v2 the pipeline reports 16 NVD-confirmed CVEs across 9 mapped components, 23 filesystem-level pattern hits, and 256 in-binary suspicious call sites — every one of which a third party can reproduce by running four scripts against the committed firmware sample.

REFERENCES

[1] OWASP Foundation, "IoT Top 10 — 2018," https://owasp.org/www-project-internet-of-things/, accessed 2026-05.

[2] M. Fagan, K. Megas, K. Scarfone, and M. Smith, "Foundational Cybersecurity Activities for IoT Device Manufacturers," NIST Internal Report 8259, 2020.

[3] European Union Agency for Cybersecurity (ENISA), "Good Practices for Security of IoT — Secure Software Development Lifecycle," 2019.

[4] OpenWrt Project, "Opkg package manager," https://openwrt.org/docs/guide-user/additional-software/opkg, accessed 2026-05.

[5] National Institute of Standards and Technology, "NVD CVE 2.0 REST API," https://nvd.nist.gov/developers/vulnerabilities, accessed 2026-05.